VEL
SOFTWARE SOLUTIONS

QA EMPLOYEE PROGRAMME

90-Day Employee Training Plan

Day-by-day onboarding and learning schedule for a beginner QA employee, with weekly deliverables, review gates and evidence checklists

Issued by **VEL Software Solutions**
velsoftwaresolutions.in@gmail.com

Document VEL-QA-002 · Version 1.0 (draft for management approval) · Issued 4 November 2026
Tool versions, licences, limits and prices change. Always verify against official documentation and vendor pricing before training, building or quoting. Confidential · internal use.

1. 90-Day Employee Training Plan

Thirteen weeks (65 working days, about 90 calendar days) take a beginner from zero to a supervised, productive QA professional. Each week has daily focus areas, a required deliverable and a review gate. Use it with the Practical Workbook.

Milestone	When	Pass requirements	Decision
M1 — Foundations	End of week 4 (~Day 30)	Level 1 and Level 2 gates passed; A0 ≥ 70; one exploratory session reported.	Continue / extend by one week with coaching
M2 — Technical testing	End of week 8 (~Day 56)	Level 3 and 4 gates; Projects 1–3 ≥ 70; can write a defect that a developer reproduces first time.	Continue / re-test weak areas
M3 — Automation & advanced QA	Week 13 (Day 90)	Level 5 and 6 gates; Projects 4–8 ≥ 70; final assessment ≥ 75.	Place as Junior QA, Manual QA, or specialist track

Weekly schedule

Week and level	Daily focus	Deliverable	Review gate
Week 1 Onboarding · Level 1a	**Mon** Day 1 agenda: policies, accounts, buddy, Golden Rules **Tue** Workstation set-up; run Practice Shop and API **Wed** SDLC, STLC, QA vs QC, verification vs validation **Thu** Errors, defects, failures; seven testing principles **Fri** Read the A0 specification; write questions and the traceability skeleton	Set-up evidence; learning journal; questions list	Buddy check Friday
Week 2 Level 1b	**Mon** Test cases and scenarios **Tue** Test plans and test data **Wed** Bug lifecycle; severity and priority **Thu** Write first 10 test cases and the first defect report **Fri** Level 1 quiz and exit-gate review	10 reviewed cases; 1 defect report; quiz	**Level 1 gate** (QA Lead)
Week 3 Level 2a	**Mon** Requirement analysis and testability checklist **Tue** Equivalence partitioning **Wed** Boundary-value analysis **Thu** Decision tables, state transitions, error guessing **Fri** A0 test design: 30+ cases and RTM	A0 suite and RTM	Peer review

Week and level	Daily focus	Deliverable	Review gate
Week 4 Level 2b	**Mon** A0 execution on Chrome and Firefox **Tue** A0 execution at 360 px; evidence and defects **Wed** Retest and A0 report **Thu** Exploratory session; smoke, sanity, regression, UAT concepts **Fri** A0 review presentation to QA Lead	A0 report; defect log; exploratory sheet	**Milestone 1 (~Day 30): A0 ≥ 70; Level 2 quiz**
Week 5 Level 3a	**Mon** Browser matrix; developer tools **Tue** Responsive testing **Wed** Form testing **Thu** Navigation and session testing **Fri** Start Project 1 (manual website test)	Browser matrix; session checklist	Buddy check
Week 6 Level 3b	**Mon** Mobile testing approaches; emulation vs real devices **Tue** OS compatibility; accessibility first-line checks **Wed** Project 1 execution **Thu** Project 1 defects and report **Fri** Level 3 gate review	Project 1 package	**Level 3 gate**; Project 1 review
Week 7 Project 2 · Level 4a	**Mon** Project 2: traceability and design **Tue** Project 2: 60+ cases **Wed** Project 2: review and fixes **Thu** HTTP methods, status codes, curl **Fri** Authentication, JSON, Postman basics	Project 2 suite; Postman collection start	Project 2 review
Week 8 Level 4b	**Mon** Schema validation and negative API tests **Tue** Project 3: collection build **Wed** Project 3: defects and escalation note **Thu** Project 3 report **Fri** Level 4 gate; Milestone 2 review	Project 3 package	**Level 4 gate; Milestone 2 (~Day 56)**
Week 9 Level 5a	**Mon** Python fundamentals 1 **Tue** Python fundamentals 2; files and JSON **Wed** pytest basics; running and reading reports **Thu** Fixtures and parametrisation **Fri** Start Project 5 (API automation)	Python exercises; first API tests	Code review
Week 10 Level 5b	**Mon** Project 5 completion and schema checks **Tue** Playwright basics, locators, auto-wait **Wed** Page Object Model **Thu** Project 4: automate 15 UI tests **Fri** Cross-browser run; traces; flakiness check	Projects 4 and 5 code	Code review (independent)
Week 11 Level 5c	**Mon** GitHub Actions CI **Tue** Project 6: regression tiers and selection rules **Wed** Project 6: run on build 1.0 and 1.1 **Thu** Project 6 report **Fri** Level 5 gate	CI link; regression report	**Level 5 gate**

Week and level	Daily focus	Deliverable	Review gate
Week 12 Level 6	**Mon** Performance concepts; workload model; plan for Project 7 **Tue** k6 / JMeter / Locust scripts; validation **Wed** Project 7 runs and analysis **Thu** Accessibility automation; database validation **Fri** Flaky management; security coordination; coverage	Performance report; a11y scan; SQL checks	**Level 6 gate**
Week 13 Capstone · Assessment	**Mon** Project 8: assemble repository and documents **Tue** Project 8: metrics and recommendation **Wed** Project 8: sign-off checklist and rehearsal **Thu** Final assessment: written paper + practical **Fri** Presentation to management; role placement decision	Complete client QA report; assessment results	**Milestone 3 (Day 90): final review**

Weekly rhythm

When	Activity	Output
Every day 09:45	Stand-up with buddy (10 min): yesterday, today, blockers.	Blockers resolved or escalated.
Every day 17:30	Learning-journal entry: what I did, learned, what I do not understand.	`journal/day-nn.md` committed.
Every Friday	Review with buddy (30 min): deliverable demo, feedback, next week.	Feedback recorded in the trainee file.
End of each level	Gate review with the QA Lead.	Signed gate record.
Every 2 weeks	One-to-one with the Test Manager about progress and wellbeing.	Notes; adjustments to plan.

If the trainee falls behind

1. Identify the gap with the buddy within two working days; check understanding, not just output.
2. Add a focused coaching block (maximum 5 working days) with a specific exercise and a re-test.
3. If the gap remains after the re-test, the QA Lead and Test Manager agree a revised plan in writing.
4. Never lower an assessment pass mark to keep a schedule.

Final evidence checklist (Day 90)

- [ ] Learning repository with 90 daily journal entries.
- [ ] All level-gate records signed.
- [ ] Projects 1–8 with scores ≥ 70 and reviewer names.
- [ ] Capstone repository complete and reproducible.
- [ ] Final assessment papers and practical marksheets filed.
- [ ] Management decision and development plan recorded.

2. Beginner Onboarding

Day-one programme for a new QA trainee. By the end of week one the trainee has a working machine, a learning repository, has run the practice targets, and has met every working agreement.

Day 1 agenda

Time	Activity	Output / evidence
09:30	Welcome, department purpose, services, who is who (QA Lead, Test Manager, Release Coordinator)	Trainee knows their buddy and escalation path
10:30	Read and sign: confidentiality, acceptable use, test-data and evidence-integrity policy	Signed policy acknowledgements filed by HR
11:30	Accounts: company e-mail, GitHub organisation invite, issue tracker, test-management tool	Access checklist ticked (see below)
13:30	Workstation setup (table below)	Screenshot of every verification command
15:00	Clone the learning repository; run the VEL Practice Shop and Practice API locally	Two screenshots: app in browser, `GET /api/health` response
16:30	Read the Golden Rules and the lifecycle overview; first 1-to-1 with the buddy	Three questions written in the learning journal

Access checklist

- [ ] Company e-mail and calendar working; two-factor authentication enabled.
- [ ] GitHub account added to the company organisation with the **QA-Trainee** team role (read on client repos, write only to the learning repository).
- [ ] Issue tracker (Jira or equivalent) account; can open the **QA-TRAINING** project.
- [ ] Test-management tool (TestRail, Zephyr, Xray or Kiwi TCMS) account in the training project.
- [ ] Password manager installed. **No** shared passwords, ever.
- [ ] Access to client systems: **none** until the Test Manager issues written authorisation naming the person, the system and the dates.

Workstation setup and verification

Install each tool from the official site, then run the verification command. Paste the output into `journal/day-01-setup.md`. Exact version numbers will differ; the major version must satisfy the “Expected” column.

Tool	Verification command	Expected output	Used for
Git	`git --version`	`git version 2.x`	Version control for all QA assets
Python	`python3 --version`	`Python 3.11` or newer	pytest, API automation, utilities
Node.js (LTS)	`node --version`	`v20.x` or newer LTS	Playwright, Newman, Lighthouse
VS Code (or equivalent)	`code --version`	A version number	Editing Markdown, code, YAML
Chrome / Edge + Firefox	Open `about:version` / `about:support`	Browser version page	Manual and cross-browser testing
Browser developer tools	Press `F12`, open **Network** tab	Requests listed when you reload a page	Evidence, API inspection

```bash
# From the qa-project-template/ folder of your learning repository
python3 -m http.server 8000 --directory practice-app    # terminal 1
python3 practice-api/server.py --port 8080               # terminal 2

# Verify (terminal 3)
curl -s http://localhost:8080/api/health
# Expected: {"status":"ok","service":"vel-practice-api"}
# Open http://localhost:8000 in the browser: the VEL Practice Shop login view appears.
```

Working agreements

Topic	Rule	Why
Evidence integrity	Never edit a screenshot, log or recording. If you must redact personal data, keep the original in the restricted evidence store and note the redaction.	Evidence that cannot be trusted is worthless to the client.
Honest reporting	Report what happened, including your own mistakes and tests you could not finish.	A hidden gap is a production risk.
Confidentiality	No client data in personal e-mail, chat tools, public AI services or public repositories.	Contractual and legal exposure.
Test data	Use synthetic data only. Production extracts and real personal data are forbidden.	Privacy law and client contract.
Authorisation	Test only what is written in the scope. Out-of-scope observations are reported, not explored.	Unauthorised testing can cause outages and legal harm.
Asking for help	Spend a maximum of 30 minutes stuck, then ask your buddy. Write down what you tried.	Speed with learning.

Topic	Rule	Why
Daily rhythm	09:45 stand-up, 17:30 learning-journal entry (what I did, what I learned, what blocks me).	Visible progress and early warning.

Naming conventions used from day one

Item	Pattern	Example
Requirement	`REQ-<nnn>`	`REQ-003`
Test scenario	`TS-<area>-<nn>`	`TS-REG-01`
Test case	`TC-<area>-<nnn>`	`TC-REG-004`
Defect	`DEF-<nnn>`	`DEF-017`
Evidence file	`<yyyy-mm-dd>_<TC-id>_<PASS|FAIL|BLOCKED>_<n>.<ext>`	`2026-11-06_TC-REG-004_FAIL_1.png`
Git branch	`qa/<initials>/<topic>`	`qa/ab/registration-tests`
Commit message	`<type>(<scope>): <summary>`	`test(registration): add boundary cases for REQ-003`

End-of-week-one checklist

- [ ] All tools verified and screenshots committed to `journal/`.
- [ ] Practice Shop and Practice API run locally; you can explain what each URL does.
- [ ] You can state the Golden Rules from memory and explain why each exists.
- [ ] Assignment A0 (next chapter) started: requirement table read, questions raised with the buddy.
- [ ] Level 1 quiz scheduled for the end of week two.

3. First Manual-Testing Assignment (A0)

Test the **VEL Practice Shop** registration and login flow against a written specification, produce a traceable test suite, execute it, log defects and report results. Time-box: **4 working days**.

> **WHY THIS ASSIGNMENT FIRST**
>
> It exercises the whole manual-testing chain in miniature: read a requirement, design cases, run them, capture evidence, report a defect, and summarise status. Every later project repeats this chain with more scale or more tooling.

Specification under test

ID	Requirement
REQ-001	Registration: **Full name** is required, 2–40 characters after trimming spaces.
REQ-002	Registration: **Email** is required and must be a valid address (`name@domain.tld`).
REQ-003	Registration: **Age** is required, a whole number from **18 to 60 inclusive**.
REQ-004	Registration: **Password** 8–20 characters with at least one digit and at least one uppercase letter.
REQ-005	Registration: **Confirm password** must match the password exactly (case-sensitive).
REQ-006	Registration: the **Terms** checkbox must be ticked.
REQ-007	A successful registration shows “Registration successful” and opens the login view.
REQ-008	Login with registered credentials opens the shop. Any failure shows the generic message “Invalid email or password”.
REQ-009	On the **3rd consecutive failed login** the account locks for 30 seconds and the message “Account locked. Try again in 30 seconds.” is shown; attempts during the lock show the same message.
REQ-010	Shop: quantity per item is a whole number **1–10**. Cart total = sum(price × quantity) shown to 2 decimals; GST 18 % shown to 2 decimals (standard rounding).
REQ-011	Session expires after **90 seconds of inactivity**: “Session expired. Please log in again.” and the login view opens.
REQ-012	Logout ends the session. Using the browser Back button afterwards must **not** show shop content.
REQ-013	The application is usable at **360 px** viewport width with no horizontal scrolling.
REQ-014	Accessibility: every input has a programmatic label; every image has alt text; all functions work by keyboard with a visible focus indicator.

Scope

In scope	Out of scope
Registration view, login view, shop view (cart, quantity, totals), session timeout, logout, viewport 360 px and 1280 px, keyboard access.	Performance, security exploitation, other devices, real e-mail delivery, payment. Report any security concern you notice to your buddy instead of probing it.

Procedure

Day	Steps	Deliverable
1	1. Read REQ-001 to REQ-014 twice; list ambiguities and unanswered questions. 2. Run the app; explore for 30 minutes without a script and note what you observe (exploratory notes). 3. Create the requirements traceability matrix with one row per requirement.	`docs/requirements-traceability.md` with all 14 rows; `docs/questions.md` ; exploratory notes
2	1. For each requirement apply **equivalence partitioning** and **boundary-value analysis** where it has ranges (REQ-001, 003, 004, 010). 2. Write at least **30 test cases** using the template: ID, title, requirement, preconditions, steps, test data, expected result, priority. 3. Include positive and negative cases; at least 6 for each of REQ-003 and REQ-004.	`test-cases/functional/registration-login.csv` (or tool export), 30+ cases, each linked to a REQ
3	1. Execute every case on Chrome and Firefox at 1280 px; REQ-013 at 360 px. 2. Record Pass / Fail / Blocked with evidence file names per the convention. 3. Raise a defect (template) the moment you confirm a failure; attempt to reproduce it three times.	Filled execution log; `evidence/` files; `defects/DEF-nnn.md` for each defect
4	1. Re-check traceability: every requirement has at least one executed case. 2. Complete the test execution report with counts, defect list, coverage and your own risk statement. 3. Present for 10 minutes to the QA Lead.	`docs/test-execution-report.md` ; review notes from the QA Lead

Worked example: one test case and one defect

Use these as the quality bar. Notice that the expected result is **observable and unambiguous**, and the defect states exactly what happened versus what was required.

Test case ID	`TC-REG-004` · Requirement: REQ-003 · Technique: boundary-value analysis · Priority: High
Title	Registration accepts age 18 (lower boundary)
Preconditions	Practice Shop open at the Register view; no account exists for `bva18@example.test` .
Steps	1. Enter name `Asha Rao` . 2. Enter email `bva18@example.test` .

	3. Enter age `18`. 4. Enter password `Passw0rdOK` and the same value in Confirm password. 5. Tick Terms. 6. Click **Register**.
Expected result	The message “Registration successful” is displayed and the Login view opens. No error is shown next to Age.
Actual result / status	Fill in during execution: **PASS** / **FAIL** / **BLOCKED** + evidence file name.

Defect ID / title	`DEF-001` · Age 18 rejected although REQ-003 allows 18 to 60 inclusive
Severity / priority	Major (valid users cannot register) / High
Environment	Practice Shop 1.0, Chrome 130 on Windows 11, viewport 1280×800, 2026-11-06
Steps to reproduce	1. Open Register view. 2. Fill all fields validly, using age `18`. 3. Tick Terms and click **Register**.
Expected	Registration succeeds (18 is the inclusive lower limit).
Actual	Age field shows an error; registration is blocked. Reproduced 3 of 3 attempts.
Evidence	`evidence/2026-11-06_TC-REG-004_FAIL_1.png`

Acceptance criteria and marking (100 points)

Criterion	Points	Full marks when
Traceability	15	All 14 requirements appear in the matrix and each links to at least one executed case.
Test design quality	25	Equivalence classes and boundaries are identified and named; cases are atomic, repeatable and have measurable expected results.
Execution discipline	20	Every case has a status and evidence; results are on two browsers; REQ-013 checked at 360 px.
Defect reports	25	Each defect is reproducible by someone else from the report alone; severity and priority are justified; no duplicates.
Report and honesty	15	Counts reconcile (pass + fail + blocked + not run = total); risks and untested areas are stated plainly.

PASS MARK: 70 / 100

Below 70 the trainee repeats the weakest part with feedback. The QA Lead keeps a hidden answer key of seeded defects (never shown to trainees). Finding more defects does not earn extra marks; **report quality** does.